CH34X Application Development Manual for Linux

Version: 1.7

Contents

1 Introduction 1
2 Supported Chips 1
2.1 CH346 1
2.2 CH347 1
2.3 CH339 2
2.4 CH341 3
2.5 CH9341 3
2.6 CH9346 3
3 API Overview 4
3.1 Device Management API 4
3.1.1 Function List 4
3.2 SPI Master API 5
3.2.1 Function List 5
3.2.2 API Usage Flow 6
3.3 SPI Slave API 6
3.3.1 Function List 6
3.3.2 API Usage Flow 7
3.4 I2C Master API 7
3.4.1 Function List 7
3.4.2 API Usage Flow 8
3.5 I2C Slave API 8
3.5.1 Function List 8
3.5.2 API Usage Flow 9
3.6 JTAG API 9
3.6.1 Function List 9
3.6.2 API Usage Flow 11
3.7 FIFO API 11
3.7.1 Function List 11
3.7.2 API Usage Flow 12
3.8 MEM/EPP API 12
3.8.1 Function List 12
3.8.2 API Usage Flow 13
3.9 PWM API 13
3.9.1 Function List 13
3.9.2 API Usage Flow 14
3.10 GPIO API 14
3.10.1 Function List 14
3.10.2 API Usage Flow 15
3.11 UART API 15

3.11.1 Function List 15
3.11.2 API Usage Flow 16
4 API Reference 16
4.1 Device Management API 16
4.1.1 CH34X_GetDeviceMode 16
4.1.2 CH34X_SetDeviceMode 17
4.1.3 CH34X_GetDeviceInfoList 17
4.1.4 CH34X_GetDeviceInfo 17
4.1.5 CH34X_GetDeviceIndex 17
4.1.6 CH34X_OpenDevice 18
4.1.7 CH34X_OpenDeviceEx 18
4.1.8 CH34X_CloseDevice 18
4.1.9 CH34X_ResetDevice 18
4.1.10 CH34X_SetDeviceNotify 19
4.1.11 CH34X_GetDriverVersion 19
4.1.12 CH34X_GetLibraryVersion 19
4.1.13 CH34X_SetUSBTimeout 19
4.2 SPI Master API 20
4.2.1 CH34X_GetInfo_SPIMaster 20
4.2.2 CH34X_SPIMasterInit 20
4.2.3 CH34X_SPISetDefaultCS 20
4.2.4 CH34X_SPIChangeCS 21
4.2.5 CH34X_SPIFastWrite 21
4.2.6 CH34X_SPIFastRead 22
4.2.7 CH34X_SPIWriteRead 23
4.2.8 CH34X_SPIReadQuery 23
4.3 SPI Slave API 24
4.3.1 CH34X_GetInfo_SPISlave 24
4.3.2 CH34X_SPISlaveInit 24
4.3.3 CH34X_SPISlaveControl 25
4.3.4 CH34X_SPISlaveWrite 25
4.3.5 CH34X_SPISlaveQueryRead 25
4.3.6 CH34X_SPISlaveRead 26
4.3.7 CH34X_SPISlaveClear 26
4.4 I2C Master API 26
4.4.1 CH34X_GetInfo_I2CMaster 26
4.4.2 CH34X_I2CMasterInit 27
4.4.3 CH34X_StreamI2C 27
4.5 I2C Slave API 28
4.5.1 CH34X_GetInfo_I2CSlave 28
4.5.2 CH34X_I2CSlaveInit 28
4.5.3 CH34X_I2CSlaveControl 28
4.5.4 CH34X_I2CSlaveWrite 29
4.5.5 CH34X_I2CSlaveClear 29
4.5.6 CH34X_I2CSlaveQueryRead 29

4.5.7 CH34X_I2CSlaveRead 29
4.6 JTAG API 30
4.6.1 CH34X_JTAGReset 30
4.6.2 CH34X_JTAGResetTRST 30
4.6.3 CH34X_JTAGInit 30
4.6.4 CH34X_JTAGGetCfg 31
4.6.5 CH34X_JTAGChangeTMSOnEdge 31
4.6.6 CH34X_JTAGIdleClock 31
4.6.7 CH34X_JTAGChangeTMS 31
4.6.8 CH34X_JTAGIOScanT 32
4.6.9 CH34X_JTAGIOScan 32
4.6.10 CH34X_JTAGWriteRead 32
4.6.11 CH34X_JTAGFastWriteRead 33
4.6.12 CH34X_JTAGSwitchTAP 33
4.6.13 CH34X_JTAGWriteDR 34
4.6.14 CH34X_JTAGReadDR 34
4.6.15 CH34X_JTAGWriteIR 34
4.6.16 CH34X_JTAGReadIR 34
4.6.17 CH34X_JTAGBitWriteDR 35
4.6.18 CH34X_JTAGBitWriteIR 35
4.6.19 CH34X_JTAGBitReadIR 35
4.6.20 CH34X_JTAGBitReadDR 36
4.7 FIFO API 36
4.7.1 CH34X_FIFOSlaveInit 36
4.7.2 CH34X_FIFOSlaveControl 37
4.7.3 CH34X_FIFOSlaveWrite 37
4.7.4 CH34X_FIFOSlaveClear 37
4.7.5 CH34X_FIFOSlaveQueryRead 37
4.7.6 CH34X_FIFOSlaveRead 38
4.8 MEM/EPP API 38
4.8.1 CH34X_SetParallelMode 38
4.8.2 CH34X_InitParallel 38
4.8.3 CH34X_EPPReadData 39
4.8.4 CH34X_EPPReadAddr 39
4.8.5 CH34X_EPPWriteData 39
4.8.6 CH34X_EPPWriteAddr 40
4.8.7 CH34X_EPPSetAddr 40
4.8.8 CH34X_MEMReadAddr0 40
4.8.9 CH34X_MEMReadAddr1 41
4.8.10 CH34X_MEMWriteAddr0 41
4.8.11 CH34X_MEMWriteAddr1 41
4.9 PWM API 42
4.9.1 CH34X_GetInfo_PWM 42
4.9.2 CH34X_PWMInit 42
4.9.3 CH34X_PWMControl 42

4.10 GPIO API 42
4.10.1 CH34X_GetInfo_GPIO 42
4.10.2 CH34X_GPIOInit 43
4.10.3 CH34X_GPIOSet 43
4.10.4 CH34X_GPIOGet 44
4.10.5 CH34X_GPIOSetIntRoutine 44
4.11 UART API 44
4.11.1 CH34X_GetInfo_UART 44
4.11.2 CH34X_UARTInit 45
4.11.3 CH34X_UARTRead 45
4.11.4 CH34X_UARTWrite 45
5 Data Types and Error Codes 47
5.1 Data Types 47
5.1.1 CH34X_DEVICE_INFO 47
5.1.2 mSPI_Master_Cfg 48
5.1.3 mSPI_Slave_Cfg 51
5.1.4 mI2C_Master_Cfg 52
5.1.5 mI2C_Slave_Cfg 54
5.1.6 mUART_Cfg 55
5.1.7 mGPIO_Cfg 56
5.1.8 mPWM_Cfg 58
5.2 Error Code 59

1 Introduction

This document provides a comprehensive introduction to the software APIs of WCH USB-to-multi-interface products. The API library offers a unified software interface for various communication and control peripherals, including UART, I2C Master/Slave, SPI Master/Slave, JTAG, FIFO, MEM/EPP parallel port, GPIO, PWM, and other supported functions. This library supports the following specific chip models: CH346C/CH347F/T/CH339W/CH341F/A/B/T/CH9341C/CH9346W and WCH USB Serial Port chip series.

This document describes the API definitions, parameter details, return values, usage procedures, and application examples. It is intended to assist developers in efficiently developing applications based on WCH USB-to-multi-interface products by providing a clear understanding of the API architecture and programming methods.

If you encounter any issues during development or require further technical assistance, please contact WCH technical support at tech@wch.cn.

2 Supported Chips

2.1 CH346

CH346 is a USB2.0 high speed converter chip to realize USB-UART, USB-SPI, USB-FIFO and USB-GPIO interfaces. CH346C supports 3 working modes, which need to be selected according to the application.

According to the characteristics of the USB converter interfaces supported by the CH346, "libch34x" provides interface functions for the parallel FIFO, SPI, and UART interfaces, including basic functional interfaces as well as the corresponding extended functional interfaces.

The interfaces supported by the CH346C are shown in the table below. Different operating modes can be switched by configuring the MODE pin level at power-up or by configuring the EEPROM.

Working Mode	Functional Interface Description	Driver Interface	API
Mode 0	Interface 0: USB2.0 to High-speed UART0	CH343SER(VCP)	Native termios API in the system or UART API in ch34x_lib.h
	Interface 1: USB2.0 to Passive Parallel FIFO	CH341PAR	FIFO Slave API in ch34x_lib.h
Mode 1	Interface 0: USB2.0 to High-speed UART0	CH343SER(VCP)	Native termios API in the system or UART API in ch34x_lib.h
	Interface 1: USB2.0 to Passive SPI interface	CH341PAR	SPI Slave in ch34x_lib.h
Mode 2	Interface 0: USB2.0 to High-speed UART0	CH343SER(VCP)	Native termios API in the system or UART API in ch34x_lib.h
	Interface 1:USB2.0 to High-speed UART1		

2.2 CH347

CH347 is a USB2.0 high-speed converter chip to realize USB to UART (HID/VCP serial port), USB to SPI, USB to I2C, USB to JTAG and USB to GPIO interfaces, CH347T supports four working modes.

According to the characteristics of the USB converter interfaces supported by the CH347, "libch34x" provides interface functions for the SPI, I2C, JTAG and UART interfaces, including the basic and extended functions, such as EEPROM read/write and SHIFT-DR state read/write in JTAG application.

The CH347F can use all interfaces without switching modes, and the supported interfaces are shown in the table below:

Functional Interface Description	Driver Interface	API
Interface0: USB2.0 to high speed UART0	CH343SER(VCP)	Native termios API in the system or UART API in ch34x_lib.h
Interface1: USB2.0 to high speed UART1		
Interface2: USB2.0 to JTAG + SPI + I2C, etc.	CH341PAR	SPI Master, I2C Master, JTAG, GPIO API in ch34x_lib.h

The following table lists the ports supported by CH347T, switching between modes via MODE configuration pin level combinations at power-on.

Working Mode	Functional Interface Description	Driver Interface	API
Mode 0	Interface 0: USB2.0 to High-speed UART0	CH343SER(VCP)	Native termios API in the system or UART API in ch34x_lib.h
	Interface 1: USB2.0 to High-speed UART1		
Mode 1	Interface 0: USB2.0 to High-speed UART1	CH343SER(VCP)	Native termios API in the system or UART API in ch34x_lib.h
	Interface 1: USB2.0 to SPI+I2C	CH341PAR	SPI Master, I2C Master, GPIO API in ch34x_lib.h
Mode 2	Interface 0: USB2.0 HID to High-speed UART1	HID driver (System-provided)	UART API in ch34x_lib.h
	Interface 1: USB2.0 HID to SPI+I2C		SPI Master, I2C Master, GPIO API in ch34x_lib.h
Mode 3	Interface 0: USB2.0 to High-speed UART1	CH343SER(VCP)	Native termios API in the system or UART API in ch34x_lib.h
	Interface 1: USB2.0 to JTAG	CH341PAR	JTAG API in ch34x_lib.h

2.3 CH339

The CH339 is a seven-port multifunction USB HUB controller chip compliant with the USB 2.0 specification. A single chip integrates a 7-port USB HUB, USB Fast Ethernet, USB high-speed SD card reader, USB Power Delivery (PD), and multiple USB conversion interfaces, including USB-to-SPI, USB-to-JTAG, USB-to-UART, and USB-to-I2C.

According to the characteristics of USB converter interface supported by CH339, "libch34x" provides

interface functions for the SPI, I2C, JTAG and UART interfaces, including the basic and extended functions.

Functional Interface Description	Driver Interface	API
USB2.0 to high speed UART	CH343SER(VCP)	Native termios API in the system or UART API in ch34x_lib.h
USB2.0 to JTAG + SPI + I2C, etc.	CH341PAR	SPI Master, I2C Master, JTAG, GPIO API in ch34x_lib.h

2.4 CH341

CH341 is a USB2.0 full-speed converter chip to realize USB to UART, USB to SPI, USB to I2C, USB to printer port and USB to EPP/MEM parallel port. CH341A/B/F can simultaneously support the above interfaces with three working modes, which need to be selected separately, CH341T supports UART and I2C interfaces.

According to the USB interface features supported by the CH341, "libch34x" provides interface functions for the SPI, I2C, EPP/MEM parallel port and UART interfaces.

SCL and SDA pin states	Driver Interface	API
Both SDA and SCL pins are in a floating state	CH341SER(VCP)	Native termios API in the system or UART API in ch34x_lib.h
SDA is connected to low level and SCL is floating or ACT is connected to a pull-down resistor	CH341PAR	SPI Master, I2C Master, MEM/EPP Parallel port, GPIO API in ch34x_lib.h
The SDA is directly connected to the SCL	USB Printer Device Class driver	

2.5 CH9341

CH9341 is a USB2.0 full-speed converter chip to realize USB to SPI, USB to I2C, USB to PWM and USB to GPIO interfaces.

According to the characteristics of the USB converter interfaces supported by the CH9341, "libch34x" provides interface functions for the SPI, I2C, PWM and GPIO interfaces.

Functional Interface Description	Driver Interface	API
USB to SPI + I2C + PWM + GPIO	CH341PAR	SPI Master/Slave, I2C Master/Slave, GPIO, PWM API in ch34x_lib.h

2.6 CH9346

CH9346 is a USB3.0 super high-speed converter chip to realize USB to FIFO, USB to UART and USB to GPIO interfaces.

According to the characteristics of the USB converter interfaces supported by the CH9346, "libch34x" provides interface functions for the FIFO and GPIO interfaces.

Working Mode	Functional Interface Description	Driver Interface	API
Mode 0	Interface 0: USB3.0 to Passive Parallel FIFO	CH341PAR	FIFO Slave API in ch34x_lib.h GPIO API in ch34x_lib.h
Mode 1	Interface 0/1: USB3.0 UVC to MEDIA interface	USB Video Device Class driver	Native API in the system
	Interface 2/3: USB3.0 CDC to UART	CDC-ACM driver (System-provided)	Native termios API in the system
Mode 2	Interface 0/1: USB3.0 UVC to DVP interface	USB Video Device Class driver	Native API in the system
	Interface 2/3: USB3.0 CDC to UART	CDC-ACM driver (System-provided)	Native termios API in the system

3 API Overview

The WCH USB-to-multi-interface API library provides a unified software interface for accessing and controlling various peripheral functions supported by WCH USB devices. The APIs are designed to simplify application development by providing consistent interfaces for device management, communication, and peripheral control.

The API library supports multiple communication interfaces and control functions, including UART, I2C Master/Slave, SPI Master/Slave, JTAG, FIFO, MEM/EPP parallel port, GPIO, PWM, and other supported features. These APIs allow developers to easily configure devices, perform data transmission, and develop applications based on WCH USB-to-multi-interface products.

The APIs are organized into different functional modules. Each module provides dedicated functions for initialization, configuration, data transfer, and device control. The following sections introduce the main API categories and their supported functionalities.

3.1 Device Management API

The Device Management APIs provide basic functions for device discovery, connection management, and device configuration. These APIs allow applications to enumerate connected WCH USB-to-multi-interface devices, retrieve device information, open and close device connections, and perform basic device operations.

The Device Management APIs support device information management, including obtaining device information, operating modes, driver and library versions. They also provide functions for device USB reset, USB hotplug notification configuration, and USB communication timeout settings.

These APIs serve as the foundation for accessing other functional modules, such as UART, I2C, SPI, JTAG, and other supported interfaces.

3.1.1 Function List

Functions	Description
CH34X_GetDeviceMode	Gets device operating mode

Functions	Description
CH34X_SetDeviceMode	Sets device operating mode
CH34X_GetDeviceInfoList	Retrieves the list of all device information in the system
CH34X_GetDeviceInfo	Retrieves information of a specified device
CH34X_GetDeviceIndex	Gets device index number from global device information array via file descriptor
CH34X_OpenDevice	Open device with device index number
CH34X_OpenDeviceEx	Open device with device information string
CH34X_CloseDevice	Close device
CH34X_ResetDevice	Perform a USB bus reset on the device
CH34X_SetDeviceNotify	USB hotplug detection function setting
CH34X_GetDriverVersion	Gets device driver version
CH34X_GetLibraryVersion	Gets library version
CH34X_SetUSBTimeout	Set the USB device read and write timeout

3.2 SPI Master API

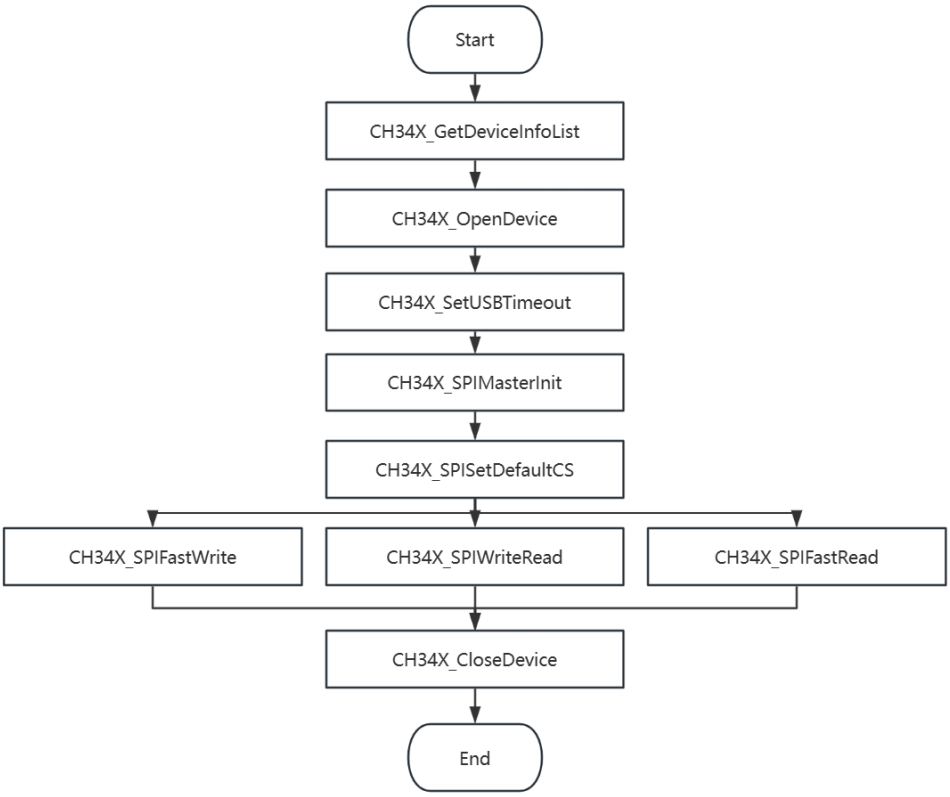
The SPI Master APIs provide functions for configuring and controlling the SPI master controller. These APIs support SPI master initialization, chip select control, SPI data read/write operations.

The SPI Master APIs include functions for SPI configuration, chip select setting, data transmission and reception.

3.2.1 Function List

Functions	Description
CH34X_GetInfo_SPIMaster	Gets device SPI master configuration
CH34X_SPIMasterInit	SPI master controller initialization
CH34X_SPISetDefaultCS	Set the default SPI chip select channel
CH34X_SPIChangeCS	Manual SPI chip select control
CH34X_SPIFastWrite	SPI fast write operation
CH34X_SPIFastRead	SPI fast read operation
CH34X_SPIWriteRead	SPI general read and write operation
CH34X_SPIReadQuery	SPI auto read operation

3.2.2 API Usage Flow



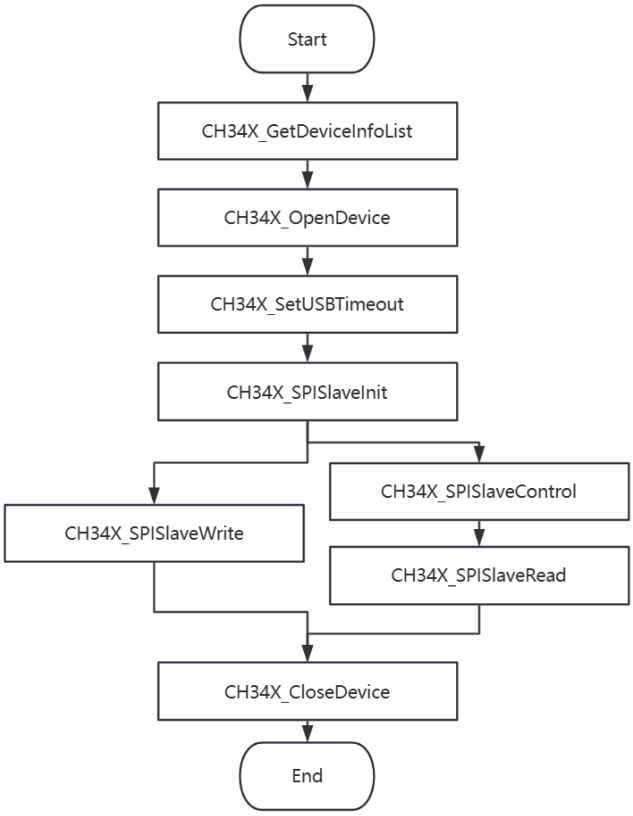
3.3 SPI Slave API

The SPI Slave APIs provide functions for configuring and controlling the SPI slave controller. These APIs support SPI slave initialization, data reading control, data writing, data querying, data reading, and data clearing operations.

3.3.1 Function List

Functions	Description
CH34X_GetInfo_SPISlave	Get SPI slave controller configuration
CH34X_SPISlaveInit	Initialize SPI slave
CH34X_SPISlaveControl	Switch of read SPI data from SPI master to chip
CH34X_SPISlaveWrite	Write data to the chip for the SPI master to read
CH34X_SPISlaveQueryRead	Query data sent by the SPI master cached in the driver
CH34X_SPISlaveRead	Read data sent by the SPI master cached in the driver
CH34X_SPISlaveClear	Clear data sent by the SPI master cached in the driver

3.3.2 API Usage Flow



3.4 I2C Master API

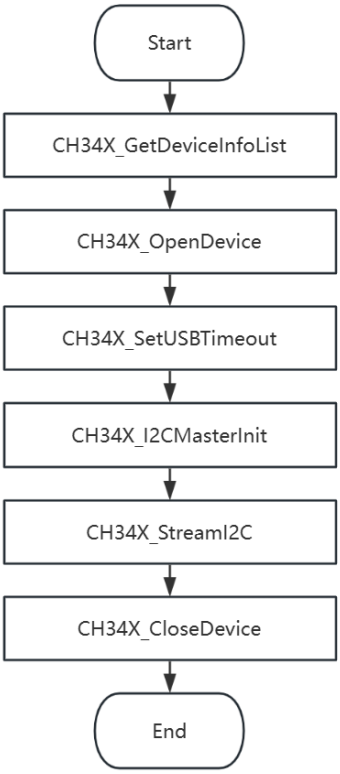
The I2C Master APIs provide functions for controlling WCH USB-to-multi-interface devices as I2C master controllers. These APIs allow applications to configure the I2C interface and communicate with external I2C slave devices.

The I2C Master APIs provide a unified interface for I2C initialization and data transfer. Applications can use these APIs to configure I2C parameters and perform read/write operations through the I2C bus.

3.4.1 Function List

Functions	Description
CH34X_GetInfo_I2CMaster	Gets device I2C master configuration
CH34X_I2CMasterInit	I2C master controller initialization
CH34X_StreamI2C	Write/Read I2C in stream mode

3.4.2 API Usage Flow



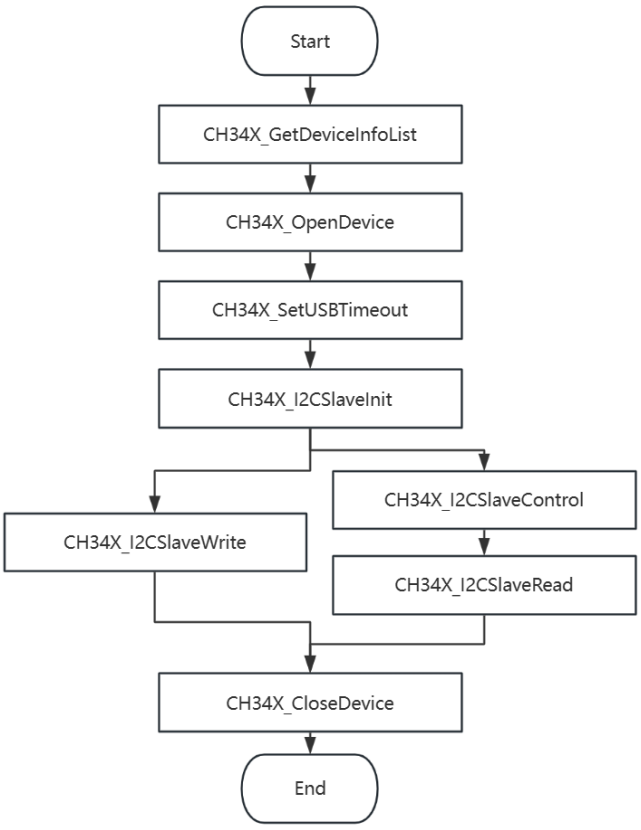
3.5 I2C Slave API

The I2C Slave APIs provide functions for configuring and controlling the I2C slave controller. These APIs support I2C slave initialization, data reading control, data writing, data querying, data reading, and data clearing operations.

3.5.1 Function List

Functions	Description
CH34X_GetInfo_I2CSlave	Gets device I2C slave configuration
CH34X_I2CSlaveInit	I2C slave controller initialization
CH34X_I2CSlaveControl	Switch of read I2C data from I2C master to chip
CH34X_I2CSlaveWrite	Write I2C data to chip via USB, the data will be cached in chip waiting I2C master to read
CH34X_I2CSlaveClear	Clear I2C data FIFO cached in the driver
CH34X_I2CSlaveQueryRead	Query the I2C data length cached in the driver
CH34X_I2CSlaveRead	Read the I2C data cached in the driver

3.5.2 API Usage Flow



3.6 JTAG API

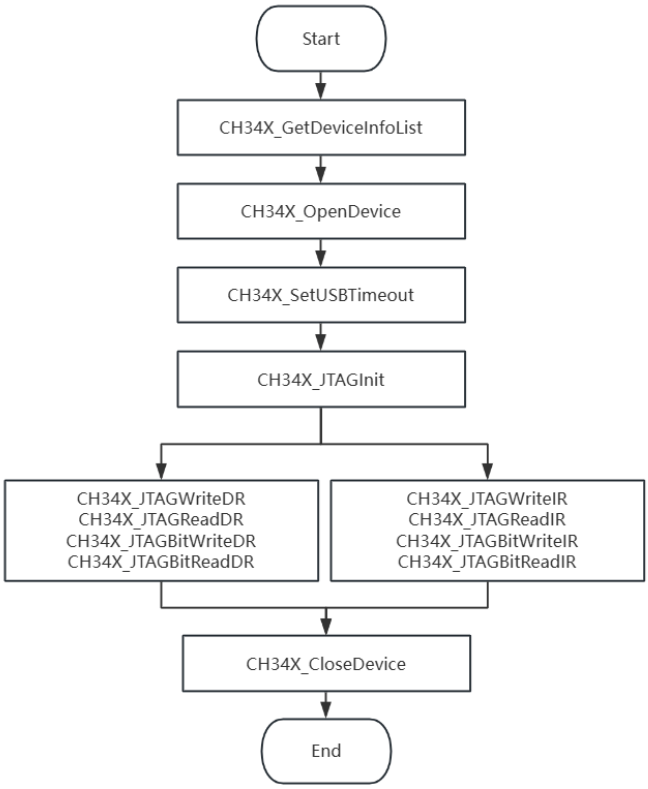
The JTAG APIs provide functions for configuring and controlling JTAG operations. These APIs support JTAG initialization, reset operations, TAP state control, JTAG data transfer, and instruction/data register access operations.

3.6.1 Function List

Functions	Description
CH34X_JTAGReset	Reset Tap Status
CH34X_JTAGResetTRST	Hard-reset JTAG device
CH34X_JTAGInit	Initialize the JTAG interface, set mode and speed
CH34X_JTAGGetCfg	Get JTAG speed setting
CH34X_JTAGChangeTMSOnEdge	Change TMS value on the rising edge of TCK to switch its Tap state
CH34X_JTAGIdleClock	Appends a JTAG idle clock state (TCK low) to the bit-bang packet
CH34X_JTAGChangeTMS	Change TMS value to switch state
CH34X_JTAGIOScanT	Read and write in the Shift-DR/IR state

Functions	Description
CH34X_JTAGIOScan	Read and write in the Shift-DR/IR state
CH34X_JTAGWriteRead	Bitband mode JTAG IR/DR data read and write which is applicable for a small amount of data
CH34X_JTAGFastWriteRead	JTAG IR/DR data read and write in batches for multi-byte continuous operation
CH34X_JTAGSwitchTAP	Switch JTAG state machine
CH34X_JTAGWriteDR	JTAG DR write in bytes which used for multi-byte continuous operation
CH34X_JTAGReadDR	JTAG DR read in bytes which used for multi-byte continuous operation
CH34X_JTAGWriteIR	JTAG IR write in bytes which used for multi-byte continuous operation
CH34X_JTAGReadIR	JTAG IR read in bytes which used for multi-byte continuous operation
CH34X_JTAGBitWriteDR	Bitband mode JTAG DR data write which is applicable for a small amount of data
CH34X_JTAGBitWriteIR	Bitband mode JTAG IR data write which is applicable for a small amount of data
CH34X_JTAGBitReadIR	Bitband mode JTAG IR data read which is applicable for a small amount of data
CH34X_JTAGBitReadDR	Bitband mode JTAG DR data read which is applicable for a small amount of data

3.6.2 API Usage Flow



3.7 FIFO API

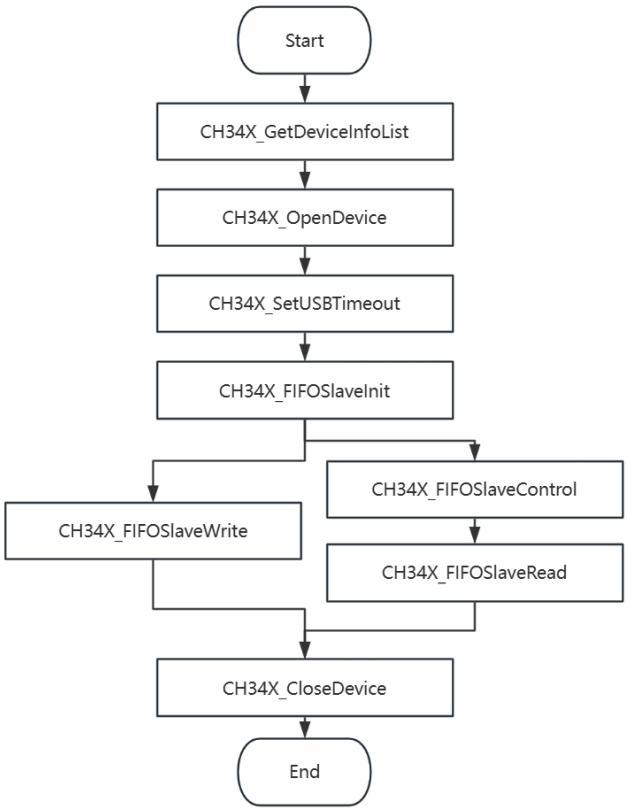
The FIFO Slave APIs provide functions for configuring and controlling the FIFO slave controller. These APIs support FIFO initialization, continuous read control, data writing, data querying, data reading, and data clearing operations.

This API module is supported by CH346 and CH9346 devices.

3.7.1 Function List

Functions	Description
CH34X_FIFOSlaveInit	FIFO slave controller initialization
CH34X_FIFOSlaveControl	Switch of read FIFO data from FIFO master to chip
CH34X_FIFOSlaveWrite	Write FIFO data to chip via USB, the data will be cached in chip waiting FIFO master to read
CH34X_FIFOSlaveClear	Clear FIFO data cached in the driver
CH34X_FIFOSlaveQueryRead	Query the FIFO data length cached in the driver
CH34X_FIFOSlaveRead	Read the FIFO data cached in the driver

3.7.2 API Usage Flow



3.8 MEM/EPP API

The MEM/EPP APIs provide functions for configuring and controlling the parallel port in MEM and EPP modes.

These APIs support parallel port mode configuration, initialization, and data read/write operations in EPP and MEM modes.

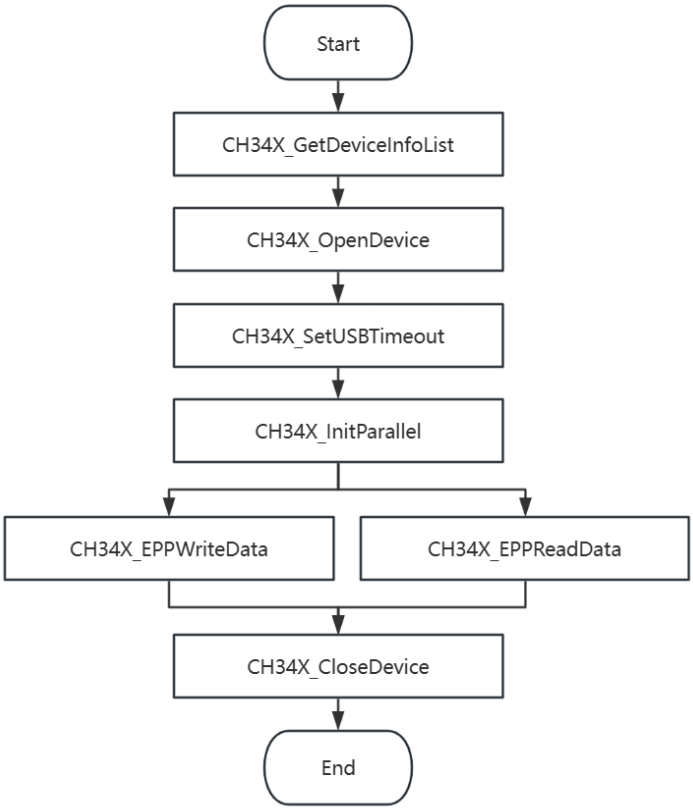
These APIs are only supported by CH341 devices.

3.8.1 Function List

Functions	Description
CH34X_SetParallelMode	Set device parallel port working mode
CH34X_InitParallel	Reset device parallel port and initialize it
CH34X_EPPReadData	Read the parallel data from device in EPP mode
CH34X_EPPReadAddr	Read the parallel address from device in EPP mode
CH34X_EPPWriteData	Write the parallel data to device in EPP mode
CH34X_EPPWriteAddr	Write the parallel address to device in EPP mode
CH34X_EPPSetAddr	Setting the parallel address in EPP mode

Functions	Description
CH34X_MEMReadAddr0	Read the parallel data from address 0 in MEM mode
CH34X_MEMReadAddr1	Read the parallel data from address 1 in MEM mode
CH34X_MEMWriteAddr0	Write the parallel data to address 0 in MEM mode
CH34X_MEMWriteAddr1	Write the parallel data to address 1 in MEM mode

3.8.2 API Usage Flow



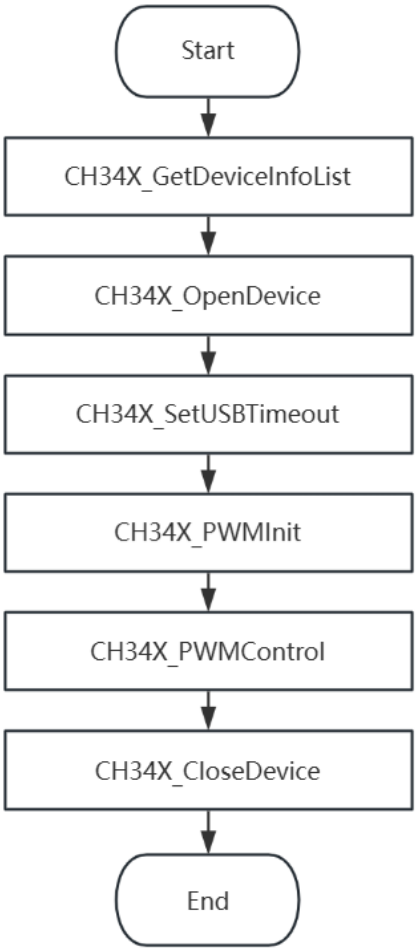
3.9 PWM API

The PWM APIs provide functions for configuring and controlling the PWM controller. These APIs support PWM initialization and PWM output control.

3.9.1 Function List

Functions	Description
CH34X_GetInfo_PWM	Gets PWM controller configuration of the specified PWM channel
CH34X_PWMInit	Initialization of the specified PWM channel
CH34X_PWMControl	Controls the start or stop of the output of the specified PWM channel

3.9.2 API Usage Flow



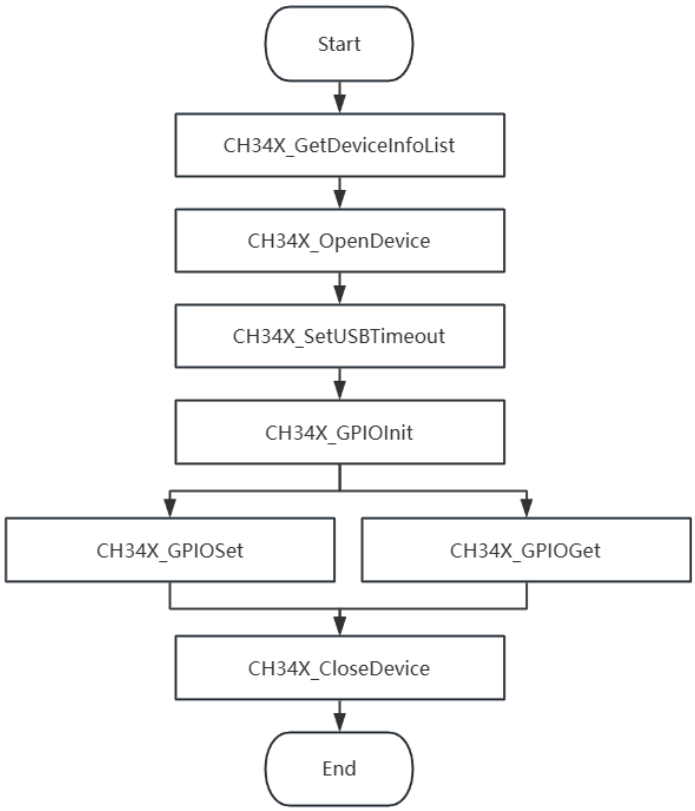
3.10 GPIO API

The GPIO APIs provide functions for configuring and controlling GPIO pins. These APIs support GPIO initialization, output level setting, and input level status reading.

3.10.1 Function List

Functions	Description
CH34X_GetInfo_GPIO	Gets all GPIO configurations for the device
CH34X_GPIOInit	GPIO initialization, supports single or multiple GPIOs
CH34X_GPIOSet	Set GPIO output, supports single or multiple GPIOs
CH34X_GPIOGet	Gets the level status of all GPIO
CH34X_GPIOSetIntRoutine	GPIO interrupt function setting

3.10.2 API Usage Flow



3.11 UART API

The UART APIs provide serial communication functions for WCH USB-to-multi-interface devices. The APIs provide a unified interface for UART configuration, data transmission, and data reception, regardless of the underlying USB communication mode.

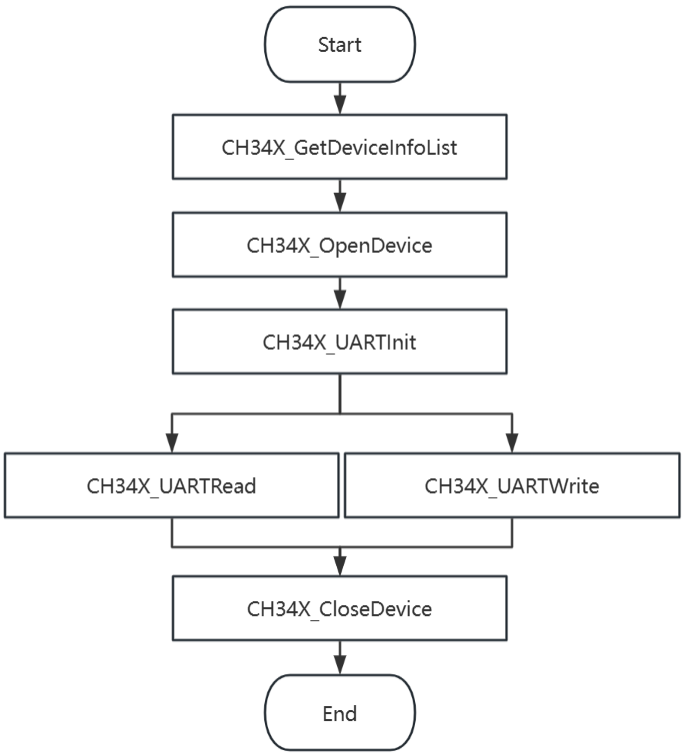
The UART APIs support both Virtual COM Port (VCP) and HID-based communication modes. The communication mode is automatically managed by the library, allowing applications to access UART functions through the same API interface.

The UART APIs include functions for retrieving UART configuration information, initializing UART, and performing UART data read/write operations.

3.11.1 Function List

Functions	Description
CH34X_GetInfo_UART	Gets configuration of UART
CH34X_UARTInit	Initialization of UART
CH34X_UARTRead	Read data from UART
CH34X_UARTWrite	Write data to UART

3.11.2 API Usage Flow



4 API Reference

This chapter provides detailed descriptions of the WCH USB-to-multi-interface APIs. Each API reference includes the function prototype, parameter descriptions, return values, and usage notes where applicable.

The APIs are organized by functional module to facilitate quick reference and application development.

4.1 Device Management API

The Device Management APIs provide basic functions for device discovery, device information retrieval, device connection management, and device configuration. These APIs support device enumeration, device information retrieval, device opening and closing, device reset, USB hotplug notification configuration, version information retrieval, and USB communication timeout configuration.

4.1.1 CH34X_GetDeviceMode

int CH34X_GetDeviceMode (int fd, uint8_t *pMode)

Function description	Gets device operating mode.
Parameter description	**fd:** File descriptor of device. **pMode:** Pointer to chip operating mode, for chips with only one operating mode, this mode value is 0.
Return value	0 if successful. Otherwise, a negative error number.

Remark	This function is only available for CH347F/T/CH339W/CH341F/A/B/T/CH9341C devices.

4.1.2 CH34X_SetDeviceMode

int CH34X_SetDeviceMode(int fd, uint8_t iMode)

Function description	Sets device operating mode.
Parameter description	**fd:** File descriptor of device. **iMode:** Chip operating mode.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346C devices.

4.1.3 CH34X_GetDeviceInfoList

int CH34X_GetDeviceInfoList(CH34X_DEVICE_INFO *pInfoList, uint8_t *pNumDevs)

Function description	Gets the information list and device count of all devices currently connected to the system, after successful execution, you can traverse each element of the array pointed to by pInfoList to obtain detailed information about each device.
Parameter description	**pInfoList:** Pointer to an array that stores all device information structures. The array must be allocated in advance by the application program. **pNumDevs:** Pointer to device count.
Return value	0 if successful. Otherwise, a negative error number.
Remark	The **CH34X_DEVICE_INFO** structure is described in detail in Chapter **Data Types and Error Codes**.

4.1.4 CH34X_GetDeviceInfo

int CH34X_GetDeviceInfo(int iIndex, CH34X_DEVICE_INFO *pInfo)

Function description	Gets the relevant information of the specified device connected to the system. When the device connection status changes, you need to call the CH34X_GetDeviceInfoList function again.
Parameter description	**iIndex:** Device index number of device list. **pInfo:** Pointer to device information structure.
Return value	0 if successful. Otherwise, a negative error number.
Remark	The **CH34X_DEVICE_INFO** structure is described in detail in Chapter **Data Types and Error Codes**.

4.1.5 CH34X_GetDeviceIndex

int CH34X_GetDeviceIndex(int fd)

Function description	Gets device index number from global device information array via file descriptor.
Parameter description	**fd:** File descriptor of device.
Return value	Device index number if successful. Otherwise, a negative error number.

4.1.6 CH34X_OpenDevice

int CH34X_OpenDevice(int iIndex)

Function description	Open device with device index number
Parameter description	**iIndex:** The index number of the connected USB device. This value can be obtained from **CH34X_GetDeviceInfoList.** Users should use the iIndex member of pInfoList.
Return value	Positive file descriptor if successful. Otherwise, a negative error number.

4.1.7 CH34X_OpenDeviceEx

int CH34X_OpenDeviceEx(CH_OPEN_TYPE type, void *pArg)

Function description	Open device with device information string
Parameter description	**type:** typedef enum { CH_OPEN_BY_SN = 0, CH_OPEN_BY_MANU_STRING, CH_OPEN_BY_PRODUCT_STRING, CH_OPEN_BY_LOCID, } CH_OPEN_TYPE; **pArg:** Pointer to USB string, whose type depends on the type parameter, must be a Pointer to a null-terminated string.
Return value	Positive file descriptor if successful. Otherwise, a negative error number.

4.1.8 CH34X_CloseDevice

int CH34X_CloseDevice(int fd)

Function description	Close device
Parameter description	**fd:** File descriptor of device
Return value	0 if successful. Otherwise, a negative error number.

4.1.9 CH34X_ResetDevice

int CH34X_ResetDevice(int fd)

Function description	Perform a USB bus reset on the device

Parameter description	**fd:** File descriptor of device
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for devices in vendor mode which named "/dev/ch34x_pis*".

4.1.10 CH34X_SetDeviceNotify

int CH34X_SetDeviceNotify(pCH_NOTIFY_ROUTINE iNotifyRoutine)

Function description	USB hotplug detection function setting
Parameter description	**iNotifyRoutine:** The handler to call when device USB hotplug event occurs, if it is NULL, the routine will be ignored.
Return value	0 if successful. Otherwise, a negative error number.
Remark	typedef void (*pCH_NOTIFY_ROUTINE)(char *Pathname, int Event); **Pathname:** Device name associated with the event. **Event:** USB event type, 0: USB remove, 1: USB insert.

4.1.11 CH34X_GetDriverVersion

int CH34X_GetDriverVersion(int fd, char *pVersion)

Function description	Gets device driver version.
Parameter description	**fd:** File descriptor of device. **pVersion:** Pointer to version string, the buffer memory size is recommended no less than 20 bytes
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for devices in vendor mode which named "/dev/ch34x_pis*".

4.1.12 CH34X_GetLibraryVersion

int CH34X_GetLibraryVersion(char *pVersion)

Function description	Gets library version.
Parameter description	**pVersion:** Pointer to version string, the buffer memory size is recommended no less than 20 bytes.
Return value	0 if successful. Otherwise, a negative error number.

4.1.13 CH34X_SetUSBTimeout

int CH34X_SetUSBTimeout(int fd, uint32_t iWriteTimeout, uint32_t iReadTimeout)

Function description	Set the USB device read and write timeout.

Parameter description	**fd:** File descriptor of device. **iWriteTimeout:** The USB write data timeout in milliseconds. 0 means no timeout, and the function blocks until the operation is completed or canceled. **iReadTimeout:** The USB read data timeout in milliseconds. 0 means no timeout, and the function blocks until the operation is completed or canceled.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is available for devices in vendor mode which named "/dev/ch34x_pis*" and devices in HID mode which named "/dev/hidraw*".

4.2 SPI Master API

4.2.1 CH34X_GetInfo_SPIMaster

int CH34X_GetInfo_SPIMaster(int fd, mSPI_Master_Cfg *pSPIMasterCfg)

Function description	Gets device SPI master configuration.
Parameter description	**fd:** File descriptor of device. **pSPIMasterCfg:** Pointer to user SPI master configuration structure.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/CH341F/A/B/CH9341C devices. The **mSPI_Master_Cfg** structure is described in detail in Chapter **Data Types and Error Codes**.

4.2.2 CH34X_SPIMasterInit

int CH34X_SPIMasterInit(int fd, mSPI_Master_Cfg *pSPIMasterCfg)

Function description	SPI master controller initialization.
Parameter description	**fd:** File descriptor of device. **pSPIMasterCfg:** Pointer to user SPI master configuration structure.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/CH341F/A/B/CH9341C devices. The **mSPI_Master_Cfg** structure is described in detail in Chapter **Data Types and Error Codes**.

4.2.3 CH34X_SPISetDefaultCS

int CH34X_SPISetDefaultCS(int fd, uint8_t iCSChannel)

Function description	Set the default SPI chip select channel, after calling API function CH34X_SPIMasterInit, this API function should be called to specify the chip select channel. After using this API function, subsequent SPI read and write functions will default to operating on this chip select channel.
Parameter description	**fd:** File descriptor of device. **iCSChannel:** SCSx channel, **0:** SCS0, **1:** SCS1, **2:** SCS2, **3:** SCS3, etc.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/CH341F/A/B/CH9341C devices.

4.2.4 CH34X_SPIChangeCS

int CH34X_SPIChangeCS(int fd, uint8_t iCSChannel, uint8_t iCSActivate)

Function description	Manual SPI chip select control, this API is suitable for applications requiring flexible chip select control or operating multiple SPI slaves. This function is optional.
Parameter description	**fd:** File descriptor of device. **iCSChannel:** SCSx channel, **0:** SCS0, **1:** SCS1, **2:** SCS2, **3:** SCS3, **iCSActivate:** Chip select activate switch, **0:** deactivate, **1:** activate.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/CH9341C devices.

4.2.5 CH34X_SPIFastWrite

int CH34X_SPIFastWrite(int fd, bool bCSAutoStart, bool bCSAutoStop, void *iWriteBuffer, uint32_t iWriteLength)

Function description	SPI fast write operation is suitable for applications that require only unidirectional write operation. The bCSAutoStart and bCSAutoStop parameters are used to control whether the SPI transfer automatically controls the chip select. When manually controlling the chip select using the CH34X_SPIChangeCS this automatic control must be disabled.
Parameter description	**fd:** File descriptor of device. **bCSAutoStart:** automatic chip select control before SPI transmission

	starts: **false:** no control, **true:** control. **bCSAutoStop:** automatic chip select control after SPI transmission ends: **false:** no control, **true:** control. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength bytes. **iWriteLength:** Number of bytes of data to be written, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/CH9341C devices.

4.2.6 CH34X_SPIFastRead

int CH34X_SPIFastRead(int fd, bool bCSAutoStart, bool bCSAutoStop, void *iWriteBuffer, uint32_t iWriteLength, void *oReadBuffer, uint32_t oReadLength)

Function description	SPI fast read operation is suitable for applications that require only unidirectional reading or mainly focus on reading data. The bCSAutoStart and bCSAutoStop parameters are used to control whether the SPI transfer automatically controls the chip select. When manually controlling the chip select using the CH34X_SPIChangeCS this automatic control must be disabled. If no data needs to be written, the parameter iWriteLength must be zero.
Parameter description	**fd:** File descriptor of device. **bCSAutoStart:** automatic chip select control before SPI transmission starts: **false:** no control **true:** control. **bCSAutoStop:** automatic chip select control after SPI transmission end: **false:** no control **true:** control. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength bytes. **iWriteLength:** Number of bytes of data to be written, valid length: 0~64KiB. **oReadBuffer:** Pointer to the receive data buffer, the buffer size of which is no less than oReadLength bytes. **oReadLength:** Number of bytes of data to be read, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.

Remark	This function is only available for CH347F/T/CH339W/CH9341C devices.

4.2.7 CH34X_SPIWriteRead

int CH34X_SPIWriteRead(int fd, bool bCSAutoStart, bool bCSAutoStop, void *ioBuffer, uint32_t ioLength)

Function description	SPI general read and write operations are suitable for general applications other than fast read and write. This function writes data from the ioBuffer buffer and writes ioLength bytes, and attempt to reads back ioLength bytes of data at the same time. After the operation is completed, the read SPI data is stored in the ioBuffer buffer. The bCSAutoStart and bCSAutoStop parameters are used to control whether the SPI transfer automatically controls the chip select. When manually controlling the chip select using the CH34X_SPIChangeCS this automatic control must be disabled.
Parameter description	**fd:** File descriptor of device **bCSAutoStart:** Automatic chip select control before SPI transmission starts, **false:** no control, **true:** control. **bCSAutoStop:** Automatic chip select control after SPI transmission ends, **false:** no control, **true:** control. **ioBuffer:** Pointer to write and read buffer, the buffer size of which is no less than ioLength bytes. **ioLength:** Number of bytes of data to be written and read, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/CH341F/A/B/CH9341C devices.

4.2.8 CH34X_SPIReadQuery

int CH34X_SPIReadQuery(int fd, bool bCSAutoStart, bool bCSAutoStop, uint16_t iCount, uint8_t iInterval, void *iWriteBuffer, uint32_t iWriteLength, void *maskBuffer, void *targetBuffer, uint32_t oReadLength)

Function description	SPI auto read operation which is suitable for applications where polling and reading are required to query the SPI slave device status, such as FLASH. This function writes data from the iWriteBuffer buffer and writes iWriteLength bytes, and reads back oReadLength bytes of data, then the read data are logically ANDed with each byte of maskBuffer. If the result matches the content of targetBuffer, the execution is successful. The chip will automatically retry iCount times at the iInterval time until the execution succeeds or times out. The bCSAutoStart and bCSAutoStop parameters are used to control whether the SPI transfer automatically controls the chip select. When

	manually controlling the chip select using the CH34X_SPIChangeCS this automatic control must be disabled
Parameter description	**fd:** File descriptor of device. **bCSAutoStart:** Automatic chip select control before SPI transmission starts: **false:** no control **true:** control. **bCSAutoStop:** Automatic chip select control after SPI transmission ends: **false:** no control **true:** control. **iCount:** SPI automatic retry times. **iInterval:** SPI automatic retry interval, unit: us. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength bytes. **iWriteLength:** Number of bytes of data to be written, valid length of iWriteLength plus oReadLength: 1~64bytes. **maskBuffer:** Pointer to mask buffer, the SPI read data will be logically ANDed with the data. **targetBuffer:** Pointer to target buffer, for comparing the data which ANDed above, when the function returns, this buffer will store the data last read from the SPI. **oReadLength:** Number of bytes of data to be read, valid length of iWriteLength plus oReadLength: 1~64bytes.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices.

4.3 SPI Slave API

4.3.1 CH34X_GetInfo_SPISlave

int CH34X_GetInfo_SPISlave(int fd, mSPI_Slave_Cfg *pSPISlaveCfg)

Function description	Gets device SPI slave configuration.
Parameter description	**fd:** File descriptor of device. **pSPISlaveCfg:** Pointer to SPI slave configuration structure.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346C/CH9341C devices. The mSPI_Slave_Cfg structure is described in detail in Chapter **Data Types and Error Codes**.

4.3.2 CH34X_SPISlaveInit

int CH34X_SPISlaveInit(int fd, mSPI_Slave_Cfg *pSPISlaveCfg)

Function description	SPI slave controller initialization.
Parameter description	**fd:** File descriptor of device. **pSPISlaveCfg:** Pointer to SPI slave configuration structure.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346C/CH9341C devices. The mSPI_Slave_Cfg structure is described in detail in Chapter **Data Types and Error Codes**.

4.3.3 CH34X_SPISlaveControl

int CH34X_SPISlaveControl(int fd, bool bEnable)

Function description	Switch of read SPI data from SPI master to chip. Once enabled, the driver will continuously read data and cache it in the FIFO.
Parameter description	**fd:** File descriptor of device. **bEnable:** Slave continuous read operation switch, **false:** disable **true:** enable
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346C/CH9341C devices.

4.3.4 CH34X_SPISlaveWrite

int CH34X_SPISlaveWrite(int fd, void *iWriteBuffer, uint32_t iWriteLength)

Function description	Write SPI data to chip, the data will be cached in chip waiting SPI master to read.
Parameter description	**fd:** File descriptor of device. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength bytes. **iWriteLength:** Number of bytes of data to be written, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346C/CH9341C devices.

4.3.5 CH34X_SPISlaveQueryRead

int CH34X_SPISlaveQueryRead(int fd, uint32_t *oReadLength)

Function description	Query the SPI data length cached in the driver
Parameter description	**fd:** File descriptor of device. **oReadLength:** Pointer to number of bytes of data cached in the driver, valid length: 0~1MiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346C/CH9341C devices.

4.3.6 CH34X_SPISlaveRead

int CH34X_SPISlaveRead(int fd, void *oReadBuffer, uint32_t *oReadLength)

Function description	Read the SPI data fifo cached in the driver, after the function executes successfully, oReadLength will store the actual data length.
Parameter description	**fd:** File descriptor of device. **oReadBuffer:** Pointer to the receive data buffer, the buffer size of which is no less than *oReadLength bytes. **oReadLength:** Pointer to number of bytes of data to be read, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346C/CH9341C devices.

4.3.7 CH34X_SPISlaveClear

int CH34X_SPISlaveClear(int fd)

Function description	Clear SPI data cached in the driver.
Parameter description	**fd:** File descriptor of device.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346C/CH9341C devices.

4.4 I2C Master API

The I2C Master APIs provide a unified interface for I2C initialization and data transfer. Applications can use these APIs to configure I2C parameters and perform read/write operations through the I2C bus.

4.4.1 CH34X_GetInfo_I2CMaster

int CH34X_GetInfo_I2CMaster(int fd, mI2C_Master_Cfg *pI2CMaster)

Function description	Gets device I2C master configuration
Parameter description	**fd:** File descriptor of device. **pI2CMaster:** Pointer to user I2C master configuration structure.

Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/ CH341F/A/B/T/CH9341C devices. The mI2C_Master_Cfg structure is described in detail in Chapter **Data Types and Error Codes.**

4.4.2 CH34X_I2CMasterInit

int CH34X_I2CMasterInit(int fd, mI2C_Master_Cfg *pI2CMasterCfg)

Function description	I2C master controller initialization
Parameter description	**fd:** File descriptor of device. **pI2CMasterCfg:** Pointer to I2C master configuration structure
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/CH341F/A/B/T/CH9341C devices. The mI2C_Master_Cfg structure is described in detail in Chapter **Data Types and Error Codes.**

4.4.3 CH34X_StreamI2C

int CH34X_StreamI2C(int fd, bool bI2CStart, bool bI2CStop, uint16_t iWriteReadDelay, void *iWriteBuffer, uint32_t iWriteLength, void *oReadBuffer, uint32_t oReadLength, uint32_t *iAckCount)

Function description	Write/Read I2C in stream mode The bI2CStart and bI2CStop parameters are used to control whether the I2C transfer automatically issue the I2C Start or Stop.
Parameter description	**fd:** File descriptor of device. **bI2CStart:** Issue I2C Start before I2C transmission starts: **false:** no control **true:** control. **bI2CStop:** Issue I2C Stop after I2C transmission ends: **false:** no control **true:** control. **iWriteReadDelay:** Interval between I2C write and read, unit: us. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength bytes. The first byte is usually the I2C device address, and the second byte is the address of the register. **iWriteLength:** Number of bytes of data to be written, valid length: 1~64KiB. **oReadBuffer:** Pointer to the receive data buffer, the buffer size of which is no less than oReadLength bytes. **oReadLength:** Number of bytes of data to be read, valid length: 0~64KiB. This parameter should be set to 0, when only data needs to be written to

	the I2C slave. **iAckCount:** Pointer to the number of ACK bits for the I2C write operation.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/ CH341F/A/B/T/CH9341C devices.

4.5 I2C Slave API

4.5.1 CH34X_GetInfo_I2CSlave

int CH34X_GetInfo_I2CSlave(int fd, mI2C_Slave_Cfg *pI2CSlave)

Function description	Gets device I2C slave configuration.
Parameter description	**fd**: File descriptor of device. **pI2CSlave**: Pointer to user I2C slave configuration structure.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices. The **mI2C_Slave_Cfg** structure is described in detail in Chapter **Data Types and Error Codes**.

4.5.2 CH34X_I2CSlaveInit

int CH34X_I2CSlaveInit(int fd, mI2C_Slave_Cfg *pI2CSlaveCfg)

Function description	I2C slave controller initialization.
Parameter description	**fd**: File descriptor of device. **pI2CSlaveCfg**: Pointer to user I2C slave configuration structure.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices. The **mI2C_Slave_Cfg** structure is described in detail in Chapter **Data Types and Error Codes**.

4.5.3 CH34X_I2CSlaveControl

int CH34X_I2CSlaveControl(int fd, bool bEnable)

Function description	Switch of read I2C data from I2C master to chip. Once enabled, the driver will continuously read data and cache it in the FIFO.
Parameter description	**fd:** File descriptor of device. **bEnable:** Slave continuous read operation switch: **false:** disable, **true:** enable.

Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices.

4.5.4 CH34X_I2CSlaveWrite

int CH34X_I2CSlaveWrite(int fd, void *iWriteBuffer, uint32_t iWriteLength)

Function description	Write I2C data to chip via USB, the data will be cached in chip waiting I2C master to read.
Parameter description	**fd:** File descriptor of device. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength bytes. **iWriteLength:** Number of bytes of data to be written, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices.

4.5.5 CH34X_I2CSlaveClear

int CH34X_I2CSlaveClear(int fd)

Function description	Clear I2C data fifo cached in the driver.
Parameter description	**fd:** File descriptor of device.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices.

4.5.6 CH34X_I2CSlaveQueryRead

int CH34X_I2CSlaveQueryRead(int fd, uint32_t *oReadLength)

Function description	Query the I2C data length cached in the driver, when using the CH34X_I2CSlaveRead interface, it is recommended to query this API before reading I2C data cached in the driver each time.
Parameter description	**fd:** File descriptor of device. **oReadLength:** Pointer to number of bytes of data cached in the driver, valid length: 0~1MiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices.

4.5.7 CH34X_I2CSlaveRead

int CH34X_I2CSlaveRead(int fd, void *oReadBuffer, uint32_t *oReadLength)

Function description	Read the I2C data cached in the driver, after the function executes successfully, oReadLength will store the actual data length.
Parameter description	**fd:** File descriptor of device. **oReadBuffer:** Pointer to the receive data buffer, the buffer size of which is no less than *oReadLength bytes. **oReadLength:** Pointer to number of bytes of data to be read, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices.

4.6 JTAG API

4.6.1 CH34X_JTAGReset

int CH34X_JTAGReset(int fd)

Function description	Reset Tap Status, more than six consecutive TCK and TMS is high will set the state machine to the Test-Logic Reset state.
Parameter description	**fd:** File descriptor of device.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.2 CH34X_JTAGResetTRST

int CH34X_JTAGResetTRST(int fd, bool TRSTLevel)

Function description	Hard-reset JTAG device.
Parameter description	**fd:** File descriptor of device. **TRSTLevel:** Reset level, false: low, true: high.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.3 CH34X_JTAGInit

int CH34X_JTAGInit(int fd, uint8_t iClockRate)

Function description	Initialize the JTAG interface, set speed.
Parameter description	**fd:** File descriptor of device. **iClockRate:** Communication speed, valid value is 0-5, the higher the value, the faster the speed.
Return value	0 if successful. Otherwise, a negative error number.

Remark	This function is only available for CH347/CH339 devices.

4.6.4 CH34X_JTAGGetCfg

int CH34X_JTAGGetCfg(int fd, uint8_t *ClockRate)

Function description	Get JTAG speed setting.
Parameter description	**fd:** File descriptor of device. **ClockRate:** Pointer to communication speed, valid value is 0-5, the higher the value, the faster the speed.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.5 CH34X_JTAGChangeTMSOnEdge

uint32_t CH34X_JTAGChangeTMSOnEdge(uint8_t *BitBangPkt, uint32_t Tms, uint32_t BI)

Function description	Change TMS value on the rising edge of TCK to switch its Tap state.
Parameter description	**BitBangPkt:** Protocol package **Tms:** TMS value to change **BI:** Length of protocol package
Return value	Length of protocol package.
Remark	This function is only available for CH347/CH339 devices.

4.6.6 CH34X_JTAGIdleClock

uint32_t CH34X_JTAGIdleClock(uint8_t *BitBangPkt, uint32_t BI)

Function description	Generates a JTAG idle clock cycle command according to the current JTAG signal states and appends the generated bit-bang data into the transmission buffer.
Parameter description	**BitBangPkt:** Protocol package **BI:** Length of protocol package
Return value	Length of protocol package.
Remark	This function is only available for CH347/CH339 devices.

4.6.7 CH34X_JTAGChangeTMS

int CH34X_JTAGChangeTMS(int fd, uint8_t *tmsValue, uint32_t Step, uint32_t Skip)

Function description	Change TMS value to switch state.
Parameter description	**fd:** File descriptor of device **tmsValue:** Pointer to TMS value, unit: byte **Step:** The valid bits which stored in tmsValue

	Skip: Valid start bit
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.8 CH34X_JTAGIOScanT

int CH34X_JTAGIOScanT(int fd, uint8_t *DataBits, uint32_t DataBitsNb, bool IsRead, bool IsLastPkt)

Function description	Read and write in the Shift-DR/IR state, if it is the last package, switch to Exit DR/IR; if not, stop at Shift-DR/IR.
Parameter description	**fd:** File descriptor of device **DataBits:** Data bits to be transmitted **DataBitsNb:** Number of bits to be transmitted **IsRead:** Whether to read data **IsLastPkt:** Whether it is the last package
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.9 CH34X_JTAGIOScan

int CH34X_JTAGIOScan(int fd, uint8_t *DataBits, uint32_t DataBitsNb, bool IsRead)

Function description	Read and write in the Shift-DR/IR state, and switch to Exit DR/IR after execution.
Parameter description	**fd:** File descriptor of device **DataBits:** Data bits to be transmitted **DataBitsNb:** Number of bits to be transmitted **IsRead:** Whether to read data
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.10 CH34X_JTAGWriteRead

int CH34X_JTAGWriteRead(int fd, bool IsDR, uint32_t iWriteBitLength, void *iWriteBitBuffer, uint32_t *oReadBitLength, void *oReadBitBuffer)

Function description	Bitband mode JTAG IR/DR data read and write which is applicable for a small amount of data. Exp: command operation, state machine switching and other control transmission. For batch data transmission, it is recommended to use CH34X_JTAGFastWriteRead. Command packets are read and written in batches in 4096 bytes. State machine: Run-Test -> Shift-IR/DR.. -> exit IR/DR -> Run-Test

Parameter description	**fd:** File descriptor of device **IsDR:** **true:** DR data read and write, **false:** IR data read and write **iWriteBitLength:** Write length **iWriteBitBuffer:** Pointer to write buffer **oReadBitLength:** Pointer to read length, returns the actual number of bytes read on success **oReadBitBuffer:** Pointer to read buffer
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.11 CH34X_JTAGFastWriteRead

int CH34X_JTAGFastWriteRead(int fd, bool IsDR, uint32_t iWriteLength, void *iWriteBuffer, uint32_t *oReadLength, void *oReadBuffer)

Function description	JTAG IR/DR data read and write in batches for multi-byte continuous operation. Exp: JTAG firmware download operation. Hardware has a 4K buffer, such as write then read, the length should not exceed 4096 bytes. The buffer size can be adjusted. State machine: Run-Test -> Shift-IR/DR.. -> exit IR/DR -> Run-Test
Parameter description	**fd:** File descriptor of device **IsDR:** **true:** DR data read and write, **false:** IR data read and write **iWriteLength:** Write length **iWriteBuffer:** Pointer to write buffer **oReadLength:** Pointer to read length, returns the actual number of bytes read on success **oReadBuffer:** Pointer to read buffer
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.12 CH34X_JTAGSwitchTAP

int CH34X_JTAGSwitchTAP(int fd, uint8_t TapState)

Function description	Switch JTAG state machine.
Parameter description	**fd:** File descriptor of device **TapState:** Machine state
Return value	0 if successful. Otherwise, a negative error number.

Remark	This function is only available for CH347/CH339 devices.

4.6.13 CH34X_JTAGWriteDR

int CH34X_JTAGWriteDR(int fd, uint32_t iWriteLength, void *iWriteBuffer)

Function description	JTAG DR write in bytes which used for multi-byte continuous operation. Exp: JTAG firmware download operation. State machine: Run-Test -> Shift-DR.. -> exit DR -> Run-Test
Parameter description	**fd:** File descriptor of device **iWriteLength:** Write length **iWriteBuffer:** Pointer to write buffer
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.14 CH34X_JTAGReadDR

int CH34X_JTAGReadDR(int fd, uint32_t *oReadLength, void *oReadBuffer)

Function description	JTAG DR read in bytes which used for multi-byte continuous operation. State machine: Run-Test -> Shift-DR.. -> exit DR -> Run-Test
Parameter description	**fd:** File descriptor of device **oReadLength:** Pointer to read length, returns the actual number of bytes read on success **oReadBuffer:** Pointer to read buffer
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.15 CH34X_JTAGWriteIR

int CH34X_JTAGWriteIR(int fd, uint32_t iWriteLength, void *iWriteBuffer)

Function description	JTAG IR write in bytes which used for multi-byte continuous operation. State machine: Run-Test -> Shift-IR.. -> exit IR -> Run-Test
Parameter description	**fd:** File descriptor of device **iWriteLength:** Write length **iWriteBuffer:** Pointer to write buffer
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.16 CH34X_JTAGReadIR

int CH34X_JTAGReadIR(int fd, uint32_t *oReadLength, void *oReadBuffer)

Function description	JTAG IR read in bytes which used for multi-byte continuous operation.

	State machine: Run-Test -> Shift-IR.. -> exit IR -> Run-Test
Parameter description	**fd:** File descriptor of device **oReadLength:** Pointer to read length, returns the actual number of bytes read on success **oReadBuffer:** Pointer to read buffer
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.17 CH34X_JTAGBitWriteDR

int CH34X_JTAGBitWriteDR(int fd, uint32_t iWriteBitLength, void *iWriteBitBuffer)

Function description	Bitband mode JTAG DR data write which is applicable for a small amount of data. Exp: command operation, state machine switching and other control transmission. For batch data transmission, it is recommended to use CH34X_JTAGWriteDR State machine: Run-Test -> Shift-DR.. -> exit DR -> Run-Test
Parameter description	**fd:** File descriptor of device **iWriteBitLength:** Write length **iWriteBitBuffer:** Pointer to write buffer
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.18 CH34X_JTAGBitWriteIR

int CH34X_JTAGBitWriteIR(int fd, uint32_t iWriteBitLength, void *iWriteBitBuffer)

Function description	Bitband mode JTAG IR data write which is applicable for a small amount of data. Exp: command operation, state machine switching and other control transmission. For batch data transmission, it is recommended to use CH34X_JTAGWriteIR State machine: Run-Test -> Shift-IR.. -> exit IR -> Run-Test
Parameter description	**fd:** File descriptor of device **iWriteBitLength:** Write length **iWriteBitBuffer:** Pointer to write buffer
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.19 CH34X_JTAGBitReadIR

int CH34X_JTAGBitReadIR(int fd, uint32_t *oReadBitLength, void *oReadBitBuffer)

Function description	Bitband mode JTAG IR data read which is applicable for a small amount of data. Exp: command operation, state machine switching and other control transmission. For batch data transmission, it is recommended to use CH34X_JTAGReadIR. State machine: Run-Test -> Shift-IR.. -> exit IR -> Run-Test
Parameter description	**fd:** File descriptor of device **oReadBitLength:** Pointer to read length, returns the actual number of bytes read on success **oReadBitBuffer:** Pointer to read buffer
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.6.20 CH34X_JTAGBitReadDR

int CH34X_JTAGBitReadDR(int fd, uint32_t *oReadBitLength, void *oReadBitBuffer)

Function description	Bitband mode JTAG DR data read which is applicable for a small amount of data. Exp: command operation, state machine switching and other control transmission. For batch data transmission, it is recommended to use CH34X_JTAGReadDR. State machine: Run-Test -> Shift-DR.. -> exit DR -> Run-Test
Parameter description	**fd:** File descriptor of device **oReadBitLength:** Pointer to read length, returns the actual number of bytes read on success **oReadBitBuffer:** Pointer to read buffer
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347/CH339 devices.

4.7 FIFO API

4.7.1 CH34X_FIFOSlaveInit

int CH34X_FIFOSlaveInit(int fd)

Function description	FIFO slave controller initialization.
Parameter description	**fd:** File descriptor of device.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346/CH9346 devices.

4.7.2 CH34X_FIFOSlaveControl

int CH34X_FIFOSlaveControl(int fd, bool bEnable)

Function description	Switch of read FIFO data from FIFO master to chip. Once enabled, the driver will continuously read data and cache it in the FIFO.
Parameter description	**fd:** File descriptor of device. **bEnable:** Slave continuous read operation switch: **false:** disable **true:** enable.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346/CH9346 devices.

4.7.3 CH34X_FIFOSlaveWrite

int CH34X_FIFOSlaveWrite(int fd, void *iWriteBuffer, uint32_t iWriteLength)

Function description	Write FIFO data to chip via USB, the data will be cached in chip waiting FIFO master to read.
Parameter description	**fd:** File descriptor of device. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength bytes. **iWriteLength:** Number of bytes of data to be written, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346/CH9346 devices.

4.7.4 CH34X_FIFOSlaveClear

int CH34X_FIFOSlaveClear(int fd)

Function description	Clear FIFO data fifo cached in the driver
Parameter description	**fd:** File descriptor of device.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346/CH9346 devices.

4.7.5 CH34X_FIFOSlaveQueryRead

int CH34X_FIFOSlaveQueryRead(int fd, uint32_t *oReadLength)

Function description	Query the FIFO data fifo length cached in the driver, when using the CH34X_FIFOSlaveRead interface, it is recommended to query this API before reading FIFO data cached in the driver each time.

Parameter description	**fd:** File descriptor of device. **oReadLength:** Pointer to number of bytes of data cached in the driver, valid length: 0~1MiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346/CH9346 devices.

4.7.6 CH34X_FIFOSlaveRead

int CH34X_FIFOSlaveRead(int fd, void *oReadBuffer, uint32_t *oReadLength)

Function description	Read the FIFO data fifo cached in the driver, after the function executes successfully, oReadLength will store the actual data length.
Parameter description	**fd:** File descriptor of device. **oReadBuffer:** Pointer to the receive data buffer, the buffer size of which is no less than *oReadLength bytes. **oReadLength:** Pointer to number of bytes of data to be read, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH346/CH9346 devices.

4.8 MEM/EPP API

4.8.1 CH34X_SetParallelMode

int CH34X_SetParallelMode(int fd, uint8_t iMode)

Function description	Set device parallel port working mode.
Parameter description	**fd:** File descriptor of device. **iMode:** Parallel port working mode, **0x00:** EPP mode V1.7 **0x01:** EPP mode V1.9 **0x02:** MEM mode
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.8.2 CH34X_InitParallel

int CH34X_InitParallel(int fd, uint8_t iMode)

Function description	Reset device parallel port and initialize it, RST# output low pulse.
Parameter description	**fd:** File descriptor of device. **iMode:** Parallel port working mode, **0x00:** EPP mode V1.7

	0x01: EPP mode V1.9 **0x02:** MEM mode.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.8.3 CH34X_EPPReadData

int CH34X_EPPReadData(int fd, void *oReadBuffer, uint32_t oReadLength)

Function description	Read the parallel data from device in EPP mode, condition: **WR#=1, DS#=0, AS#=1, D0-D7=input**.
Parameter description	**fd:** File descriptor of device. **oReadBuffer:** Pointer to the receive data buffer, the buffer size of which is no less than oReadLength bytes. **oReadLength:** Number of bytes of data to be read, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.8.4 CH34X_EPPReadAddr

int CH34X_EPPReadAddr(int fd, void *oReadBuffer, uint32_t oReadLength)

Function description	Read the parallel address from device in EPP mode, condition: **WR#=1, DS#=1, AS#=0, D0-D7=input.**
Parameter description	**fd:** File descriptor of device. **oReadBuffer:** Pointer to the receive data buffer, the buffer size of which is no less than oReadLength bytes. **oReadLength:** Number of bytes of data to be read, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.8.5 CH34X_EPPWriteData

int CH34X_EPPWriteData(int fd, void *iWriteBuffer, uint32_t iWriteLength)

Function description	Write the parallel data to device in EPP mode, condition: WR#=0, DS#=0, AS#=1, D0-D7=output.
Parameter description	**fd:** File descriptor of device. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength bytes. **iWriteLength:** Number of bytes of data to be written, valid length: 1~64KiB.

Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.8.6 CH34X_EPPWriteAddr

int CH34X_EPPWriteAddr(int fd, void *iWriteBuffer, uint32_t iWriteLength)

Function description	Write the parallel address to device in EPP mode, condition: **WR#=0, DS#=1, AS#=0, D0-D7=output**.
Parameter description	**fd:** File descriptor of device. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength bytes. **iWriteLength:** Number of bytes of data to be written, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.8.7 CH34X_EPPSetAddr

int CH34X_EPPSetAddr(int fd, uint8_t iAddress)

Function description	Setting the parallel address in EPP mode, condition: WR#=0, DS#=1, AS#=0, D0-D7=output.
Parameter description	**fd:** File descriptor of device. **iAddress:** Address in EPP mode.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.8.8 CH34X_MEMReadAddr0

int CH34X_MEMReadAddr0(int fd, void *oReadBuffer, uint32_t oReadLength)

Function description	Read the parallel data from address 0 in MEM mode, condition: **WR#=1, DS#/RD#=0, AS#/ADDR=0, D0-D7=input**.
Parameter description	**fd:** File descriptor of device. **oReadBuffer:** Pointer to the receive data buffer, the buffer size of which is no less than oReadLength bytes. **oReadLength:** Number of bytes of data to be read, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.8.9 CH34X_MEMReadAddr1

int CH34X_MEMReadAddr1(int fd, void *oReadBuffer, uint32_t oReadLength)

Function description	Read the parallel data from address 1 in MEM mode, condition: **WR#=1, DS#/RD#=0, AS#/ADDR=1, D0-D7=input**.
Parameter description	**fd:** File descriptor of device. **oReadBuffer:** Pointer to the receive data buffer, the buffer size of which is no less than oReadLength bytes. **oReadLength:** Number of bytes of data to be read, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.8.10 CH34X_MEMWriteAddr0

int CH34X_MEMWriteAddr0(int fd, void *iWriteBuffer, uint32_t iWriteLength)

Function description	Write the parallel data to address 0 in MEM mode, condition: **WR#=0, DS#/RD#=1, AS#/ADDR=0, D0-D7=output**.
Parameter description	**fd**: File descriptor of device. **iWriteBuffer**: Pointer to write buffer, the buffer size of which is no less than iWriteLength byte **iWriteLength**: Number of bytes of data to be written, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.8.11 CH34X_MEMWriteAddr1

int CH34X_MEMWriteAddr1(int fd, void *iWriteBuffer, uint32_t iWriteLength)

Function description	Write the parallel data to address 1 in MEM mode, condition: **WR#=0, DS#/RD#=1, AS#/ADDR=1, D0-D7=output**.
Parameter description	**fd:** File descriptor of device. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength byte **iWriteLength:** Number of bytes of data to be written, valid length: 1~64KiB.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH341F/B/A devices.

4.9 PWM API

4.9.1 CH34X_GetInfo_PWM

int CH34X_GetInfo_PWM(int fd, uint8_t iPWMChannel, mPWM_Cfg *pPWMCfg)

Function description	Gets PWM controller configuration of the specified PWM channel.
Parameter description	**fd:** File descriptor of device. **iPWMChannel:** PWM channel number. **pPWMCfg:** Pointer to PWM user configuration structure.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices. The mPWM_Cfg structure is described in detail in Chapter **Data Types and Error Codes**.

4.9.2 CH34X_PWMInit

int CH34X_PWMInit(int fd, uint8_t iPWMChannel, mPWM_Cfg *pPWMCfg)

Function description	Initialization of the specified PWM channel.
Parameter description	**fd:** File descriptor of device. **iPWMChannel:** PWM channel number. **pPWMCfg:** Pointer to PWM user configuration structure.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices. The mPWM_Cfg structure is described in detail in Chapter **Data Types and Error Codes**.

4.9.3 CH34X_PWMControl

int CH34X_PWMControl(int fd, uint8_t iPWMChannel, bool bStart)

Function description	Controls the start or stop of the output of the specified PWM channel.
Parameter description	**fd:** File descriptor of device. **iPWMChannel:** PWM channel number. **bStart:** PWM output control switch: **false:** stop **true:** start
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH9341C devices.

4.10 GPIO API

4.10.1 CH34X_GetInfo_GPIO

int CH34X_GetInfo_GPIO(int fd, mGPIO_Cfg *pGPIOCfgArray)

Function description	Gets all GPIO configurations for the device.
Parameter description	**fd:** File descriptor of device. **pGPIOCfgArray:** Pointer to GPIO configurations structure array.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/CH341F/A/B/CH9341C/CH9346W devices. The mGPIO_Cfg structure is described in detail in Chapter **Data Types and Error Codes**.

4.10.2 CH34X_GPIOInit

int CH34X_GPIOInit(int fd, uint64_t iGPIOChannel, mGPIO_Cfg *pGPIOCfgArray)

Function description	GPIO initialization, supports single or multiple GPIOs.
Parameter description	**fd:** File descriptor of device. **iGPIOChannel:** GPIO channel, BITx represents GPIOx, where x represents the channel number. BITx is: **0:** ignore **1:** set. **pGPIOCfgArray:** Pointer to GPIO configurations structure array.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/CH341F/A/B/CH9341C/CH9346W devices. The mGPIO_Cfg structure is described in detail in Chapter **Data Types and Error Codes**.

4.10.3 CH34X_GPIOSet

int CH34X_GPIOSet(int fd, uint64_t iGPIOChannel, uint64_t iGPIOLevel)

Function description	Controls GPIO output high or low level, supports single or multiple GPIOs.
Parameter description	**fd:** File descriptor of device. **iGPIOChannel:** GPIO channel, BITx represents GPIOx, where x represents the channel number. BITx is: **0:** ignore **1:** set **iGPIOLevel:** GPIO output level, BITx represents GPIOx, where x represents the channel number. BITx is: **0:** low **1:** high
Return value	0 if successful. Otherwise, a negative error number.

Remark	This function is only available for CH347F/T/CH339W/CH341F/A/B/CH9341C/CH9346W devices.

4.10.4 CH34X_GPIOGet

int CH34X_GPIOGet(int fd, uint64_t *iGpioLevel)

Function description	Gets the level status of all GPIO.
Parameter description	**fd:** File descriptor of device. **iGPIOLevel:** Pointer to GPIO level, BITx represents GPIOx, where x represents the channel number. BITx is: **0:** low **1:** high
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH339W/CH341F/A/B/CH9341C/CH9346W devices.

4.10.5 CH34X_GPIOSetIntRoutine

int CH34X_GPIOSetIntRoutine(int fd, uint8_t iGPIOIndex, pCH_INT_ROUTINE iIntRoutine)

Function description	GPIO interrupt function setting.
Parameter description	**fd:** File descriptor of device. **iGPIOIndex:** Device GPIO index number, 0 ⩽ iGPIOIndex < MaxGPIOChannel of the chip. **iIntRoutine:** handler to call when interrupt occurs, if it is NULL, indicates that the interrupt function has been cancelled.
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347F/T/CH9341C devices. typedef void (*pCH_INT_ROUTINE) (int SigNum); **SigNum:** Signal number SIGIO.

4.11 UART API

The UART APIs provide functions for UART configuration, initialization, and data communication. These APIs support UART information retrieval, UART initialization, and UART data read/write operations.

4.11.1 CH34X_GetInfo_UART

int CH34X_GetInfo_UART(int fd, mUART_Cfg *pUARTCfg)

Function description	Gets configuration of UART.
Parameter description	**fd:** File descriptor of device. **pUARTCfg:** Pointer to UART user configuration structure.
Return value	0 if successful. Otherwise, a negative error number.

Remark	This function is only available for CH347T in HID mode which named "/dev/hidraw*" and CH34X series in TTY mode which named "/dev/ttyCH*". The **mUART_Cfg** structure is described in detail in Chapter **Data Types and Error Codes**.

4.11.2 CH34X_UARTInit

int CH34X_UARTInit(int fd, mUART_Cfg *pUARTCfg)

Function description	Initialization of UART.
Parameter description	**fd:** File descriptor of device. **pUARTCfg:** Pointer to UART user configuration structure
Return value	0 if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347T in HID mode which named "/dev/hidraw*" and CH34X series in TTY mode which named "/dev/ttyCH*". The mUART_Cfg structure is described in detail in Chapter **Data Types and Error Codes.**

4.11.3 CH34X_UARTRead

int CH34X_UARTRead(int fd, void *oReadBuffer, uint32_t oReadLength)

Function description	Read data from UART.
Parameter description	**fd:** File descriptor of device. **oReadBuffer:** Pointer to the receive data buffer, the buffer size of which is no less than oReadLength bytes. **oReadLength:** Number of bytes of data to be read, valid length: 1~64KiB.
Return value	The actual length if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347T in HID mode which named "/dev/hidraw*" and CH34X series in TTY mode which named "/dev/ttyCH*".

4.11.4 CH34X_UARTWrite

int CH34X_UARTWrite(int fd, void *iWriteBuffer, uint32_t iWriteLength)

Function description	Write data to UART.
Parameter description	**fd:** File descriptor of device. **iWriteBuffer:** Pointer to write buffer, the buffer size of which is no less than iWriteLength bytes. **iWriteLength:** Number of bytes of data to be written, valid length: 1~64KiB.

Return value	The actual length if successful. Otherwise, a negative error number.
Remark	This function is only available for CH347T in HID mode which named "/dev/hidraw*" and CH34X series in TTY mode which named "/dev/ttyCH*".

5 Data Types and Error Codes

5.1 Data Types

5.1.1 CH34X_DEVICE_INFO

Field	Type	Description
manufacturer	char [32]	iManufacturer string
product	char [32]	iProduct string
serial	char [32]	iSerialNumber string
location	char [32]	Device USB location string
pathname	char [32]	Device path string
device_desc	struct usb_device_descriptor	USB device descriptor
iIndex	int	Device index number in list
ChipType	CHIP_TYPE	Device chip type
FuncType	CH347FUNCTYPE	CH347 Device function type
USBType	USBTYPE	USB device type
ChipMode	uint8_t	Device chip mode
fd	int	Device file descriptor
fwVer	uint16_t	Device firmware version
MaxSPIFreq	uint32_t	Maximum SPI frequency supported by the device

Field	Type	Description
MaxSCSChannel	uint8_t	Maximum SPI SCS channel supported by the device
MaxUARTChannel	uint8_t	Maximum UART channel supported by the device
MaxGPIOChannel	uint8_t	Maximum GPIO channel supported by the device
MaxPWMChannel	uint8_t	Maximum PWM channel supported by the device
SPI_DefaultCS	uint8_t	SPI default chip select

5.1.2 mSPI_Master_Cfg

This structure is used to configure SPI master parameters when calling SPI-related APIs.

Field	Type	Description	CH347	CH339	CH341	CH9341
SPI_Clock	uint32_t	SPI clock frequency	Maximum: 60M Remark: 60M/48M/36M/30M/28M/24M/18M/15M/14M/12M/9M/7.5M/7M/6M/4.5M/3.75M/3.5M/3M/2.25M/1.875M/1.75M/1.5M etc. (Unit: Hz)	Maximum: 60M Remark: 60M/30M/15M/7.5M/3.75M/1.875M etc. (Unit: Hz)	Fixed: 2M Remark: Regardless of how this parameter is set, the chip always operates at a frequency close to 2M (Unit: Hz)	Maximum: 24M Remark: Supports frequencies of 96 MHz divided by N, where N can range from 4 to 255
SPI_OutDefaultData	uint8_t	SPI master default transmit bytes	0x00-0xFF	0x00-0xFF	0xFF	0x00-0xFF

Field	Type	Description	CH347	CH339	CH341	CH9341
SPI_Mode	uint8_t	SPI mode selection: **0:** Mode 0 **1:** Mode 1 **2:** Mode 2 **3:** Mode 3	0/1/2/3	0/1/2/3	3	0/3
SPI_Enable_Line3	uint8_t	SPI data line mode: **0:** 4-line SPI **1:** 3-line SPI	0	0	0	0
SPI_DataBits	uint8_t	SPI data width: **0:** 8-bit data **1:** 16-bit data	0/1	0	0	0/1
SPI_BitOrder	uint8_t	SPI bit order: **0:** MSB first **1:** LSB first	0/1	0/1	0/1	0/1
SPI_CS0Init	uint8_t	Initialize SPI chip select SCS0: **0:** no initialization **1:** initialize to default polarity	0/1	0/1	0	0/1
SPI_CS1Init	uint8_t	Initialize SPI chip select SCS1: **0:** no initialization **1:** initialize to default polarity	0/1		0	0/1
SPI_CS2Init	uint8_t	Initialize SPI chip select SCS2: **0:** no initialization **1:** initialize to default polarity			0	0/1

Field	Type	Description	CH347	CH339	CH341	CH9341
SPI_CS3Init	uint8_t	Initialize SPI chip select SCS3: **0:** no initialization **1:** initialize to default polarity			0	0/1
SPI_CS0Polar	uint8_t	Polarity of SPI chip select SCS0: **0:** active low **1:** active high	0/1	0/1	0	0/1
SPI_CS1Polar	uint8_t	Polarity of SPI chip select SCS1: **0:** active low **1:** active high	0/1		0	0/1
SPI_CS2Polar	uint8_t	Polarity of SPI chip select SCS2: **0:** active low **1:** active high			0	0/1
SPI_CS3Polar	uint8_t	Polarity of SPI chip select SCS3: **0:** active low **1:** active high			0	0/1
SPI_DataToDataDelay	uint16_t	Delay between two consecutive SPI data bytes, unit: us	0x00-0xFFFF	0x00-0xFFFF	0x00	0x00-0xFFFF
SPI_CsToDataDelay	uint16_t	Delay between chip select	0x00-0xFFFF	0x00-0xFFFF	0x00	0x00-0xFFFF

Field	Type	Description	CH347	CH339	CH341	CH9341
		assertion and data start, unit: us				
SPI_DataToCsDelay	uint16_t	Delay between data transmission completion and chip select deassertion, unit: us	0x00-0xFFFF	0x00-0xFFFF	0x00	0x00-0xFFFF

5.1.3 mSPI_Slave_Cfg

This structure is used to configure SPI slave parameters when calling SPI-related APIs

Field Name	Type	Description	CH346	CH9341
SPI_Enable_Line3	uint8_t	SPI data line configuration. Available options: **0:** 4-line SPI **1:** 3-line SPI.	0	0
SPI_DataBits	uint8_t	SPI data bit width configuration. Available options: **0:** 8-bit data **1:** 16-bit data.	0	0/1
SPI_BitOrder	uint8_t	SPI data transmission bit order. Available options: **0:** MSB first **1:** LSB first	0	0/1
SPI_CS0Polar	uint8_t	SPI chip select (SCS0) polarity configuration. Available options: **0:** active low	0	0/1

Field Name	Type	Description	CH346	CH9341
		1: active high		
SPI_Slave_Read_Time Out	uint8_t	Receive timeout setting in SPI slave mode. Unit: 100 us	0	0x00-0xFF

5.1.4 mI2C_Master_Cfg

This structure is used to configure I2C master parameters when calling I2C-related APIs.

Field Name	Type	Description	CH347	CH341	CH9341
I2C_Clock	uint32_t	I2C clock frequency configuration. The clock frequency should not exceed 1 MHz	Maximum: 1M Remark: 1M/750K/400K/ 200K/100K/50K/ 20K(Unit: Hz)	Maximum: 750K Remark: 750K/400K/200K/ 100K/50K/20K (Unit: Hz)	Maximum: 750K Remark: 750K/400K/200K/ 100K/50K/20K (Unit: Hz)
I2C_10BitAddr	uint8_t	I2C address mode selection. Available options: **0:** 7-bit address **1:** 10-bit address	0 Remark: The chip can still operate 10-bit devices while in 7-bit addressing mode	0 Remark: The chip can still operate 10-bit devices while in 7-bit addressing mode	0/1
I2C_Enable_SoftMode	uint8_t	Enable I2C software driver mode **0:** hardware I2C **1:** software I2C	0/1	0	0
I2C_Enable_SMBus	uint8_t	Enable SMBus function, **0:** I2C function	0	0	0

Field Name	Type	Description	CH347	CH341	CH9341
		1: SMBus function			
I2C_Disable_Stretch	uint8_t	I2C clock stretching control. Available options: **0:** clock stretching enabled **1:** clock stretching disabled	0/1	1	0/1
I2C_Send_ACK_OnReadOver	uint8_t	ACK behavior when reading the last byte of data. Available options: **0:** send NACK **1:** send ACK	0	0	0/1
I2C_Stretch_Timeout	uint8_t	SCL clock stretching detection timeout setting. Unit: 100 us	0x00-0xFF		0x00-0xFF
I2C_DutyCycle	uint8_t	I2C clock duty cycle configuration (Tlow/Thigh). Available options: **0:** 16/9 **1:** 2/1			0/1
I2C_DriverMode	uint8_t	I2C signal drive mode selection, **0:** open-drain **1:** push-pull	0/1	0	0
I2C_Continue_OnNACK	uint8_t	Control behavior when a device NACK is detected during transmission. Available options:	0/1	1	0/1

Field Name	Type	Description	CH347	CH341	CH9341
		0: stop transmission **1:** continue transmission			
I2C_WriteDelay_9thClock	uint32_t	Write delay setting between the 8th clock edge and the 9th clock edge during I2C write operations. Unit: us	0-0x3FF	0	0
I2C_ReadDelay_9thClock	uint32_t	Read delay setting between the 8th clock edge and the 9th clock edge during I2C read operations. Unit: us	0-0x3FF	0	0

5.1.5 mI2C_Slave_Cfg

This structure is used to configure I2C slave parameters when calling I2C-related APIs

Field Name	Type	Description	CH9341
I2C_Slave_Read_TimeOut	uint8_t	Receive timeout setting in I2C slave mode. unit: 100 us	0x00-0xFF
I2C_AddrMode	uint8_t	I2C slave address mode selection, **0**: 7-bit address mode **1**: 10-bit address mode	0/1
I2C_Slave_Addr	uint16_t	I2C slave device address used for address matching in slave mode	0x01-0x3FF
I2C_Slave_Reg	uint8_t	I2C slave internal data register used for data access during I2C transactions	0x00-0xFF

5.1.6 mUART_Cfg

This structure is used to configure UART parameters when calling UART-related APIs.

Field Name	Type	Description	CH347	CH346	CH339	CH341
UART_Channel	uint8_t	Indicates UART channel number, read-only.	0/1	0/1	0	1
UART_BaudRate	uint32_t	UART communication baud rate configuration	1200-9Mbps	110-15Mbps	2400-6Mbps	50-2Mbps
UART_StopBits	uint8_t	UART stop bit configuration, **0:** 1 stop bit **1:** 1.5 stop bits **2:** 2 stop bits	0/2	0/2	0/2	0/2
UART_Parity	uint8_t	UART parity mode selection, **0:** none **1:** odd **2:** even **3:** mark **4:** space	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4
UART_DataBits	uint8_t	UART data bit length selection, **5:** 5 bits **6:** 6 bits **7:** 7 bits **8:** 8 bits **16:** 16 bits	5/6/7/8	5/6/7/8	5/6/7/8	5/6/7/8
UART_FlowControl	uint8_t	UART flow control mode,	0/1	0/1	0/1	0/1

Field Name	Type	Description	CH347	CH346	CH339	CH341
		0: none **1:** hardware flow control **2:** software flow control				
UART_ReceiveTimeout	uint32_t	UART receive timeout setting. unit: 100ms.	0x00-0xFF	0x00-0xFF	0x00-0xFF	0x00-0xFF

5.1.7 mGPIO_Cfg

This structure is used to configure GPIO parameters when calling GPIO-related APIs.

Field Name	Type	Description	CH347	CH341	CH339	CH9341	CH9346
GPIO_Channel	uint8_t	GPIO channel number setting	0-7	0-14	0-2	0-9	0-6
GPIO_Direction	uint8_t	GPIO direction configuration, **0:** input **1:** output	0/1	0/1	0/1	0/1	0/1
GPIO_Mode	union	GPIO mode configuration. When configured as input, this field defines the input mode, **0:** floating **1:** pull-up input **2:** pull-down input when configured as output, this field defines the output mode, **0:** push-pull output **1:** open-drain output	input: 1 output: 0	input: 1 output: 0	input: 1 output: 0	input: 0/1/2 output: 0/1	input: 0/1/2 output: 0

Field Name	Type	Description	CH347	CH341	CH339	CH9341	CH9346
GPIO_PinLevel	uint8_t	GPIO output pin level state. **0:** low level **1:** high level	0/1	0/1	0/1	0/1	0/1
GPIO_WakeUpEnable	uint8_t	GPIO wake-up function enable control, **0:** disabled **1:** enabled				0/1	
GPIO_InterruptEnable	uint8_t	GPIO interrupt function enable control	0/1 Remark: GPIO that support interrupts:0-7, interrupts on GPIO0 and GPIO1 cannot be enabled simultaneously			0/1 Remark: GPIO that support interrupts:0/1/2/7/8/9	
GPIO_InterruptMode	uint8_t	GPIO interrupt trigger mode selection **0:** falling edge **1:** rising edge **2:** low level **3:** high level **4:** both edges	0/1/2/3/4			0/1/2/3	

Field Name	Type	Description	CH347	CH341	CH339	CH9341	CH9346
GPIO_FilterDelay	uint8_t	GPIO input filter delay time in us. This setting applies to all input GPIOs simultaneously				0-255	
GPIO_DefaultEnable	uint8_t	GPIO power-down retention control **0:** ignore this setting **1:** enable power-down retention **2:** disable power-down retention				0/1/2	0/1/2

5.1.8 mPWM_Cfg

This structure is used to configure PWM parameters when calling PWM-related APIs.

Field Name	Type	Description	CH9341
PWM_Channel	uint8_t	PWM channel number	0/1
PWM_Polarity	uint8_t	PWM output polarity configuration **0:** active low **1:** active high	0/1
PWM_Start	uint8_t	PWM output enable control **0:** PWM output disabled **1:** PWM output enabled	0/1
PWM_Cycle	uint32_t	PWM output cycle count setting PWM output time = cycle number * clock cycle, CH9341 clock	0-maximum of uint32_t

		cycle: 1/96 us	
PWM_DutyCycle	uint32_t	PWM output duty cycle setting, duty cycle = PWM_DutyCycle/ PWM_Cycle	0-maximum of PWM_Cycle
PWM_RepeatTimes	uint16_t	PWM output repeat count setting. A value of 0 indicates continuous output	0-maximum of uint16_t

Note: If a parameter description is left blank, it means the chip does not support that parameter setting; any value assigned to it will have no effect. However, it is recommended to set it to 0 by default.

5.2 Error Code

Error Code	Macro Name	Description
0	ERR_SUCCESS	Operation successful
1	ERR_INVALID_HANDLE	No such device or device is not open
2	ERR_INVALID_PARAM	Invalid argument
3	ERR_MALLOC	Out of memory
4	ERR_NULLPTR	Null pointer error
5	ERR_DEVICE_OPEN	Open device error
6	ERR_DEVICE_CLOSE	Close device error
10	ERR_INVALID_CMD	Device response command error
11	ERR_PROTOCOL	Device response error

12	ERR_AGAIN	Device busy or operation timeout
13	ERR_INCOMPATIBLE	Device incompatible with this API
20	ERR_USB_COMM	USB communication error
21	ERR_USB_TIMEOUT	USB communication timeout
22	ERR_USBIN_INCOMPLETE	USB data read length not as expected
23	ERR_USB_NOFITY	USB hotplug notification setting error
30	ERR_ROLE_MS	Interface master/slave role error
31	ERR_SPI_NOTINIT	SPI interface not initialized
40	ERR_I2C_NOTINIT	I2C interface not initialized
41	ERR_I2C_NACK	I2C Device NACK Error
42	ERR_I2C_WRITE	I2C write operation error or timeout
43	ERR_I2C_READ	I2C read operation error or timeout
44	ERR_I2C_ADDRESS	I2C address error or timeout
50	ERR_GPIO_PROPERTY	GPIO property setting error
51	ERR_GPIO_SET	GPIO setting error

The table lists positive error constants defined in ch34x_lib.h. APIs documented as returning a negative error number use a negative value on failure; do not compare such a return value directly with a positive error constant. For example, if ERR_INVALID_PARAM is defined as 2 and the API interface returns -2, the check should use `ret == -ERR_INVALID_PARAM`.

UART read/write, device open/index, and JTAG packet-building APIs have different success return values; see their individual descriptions.